1. Core interaction flow

Setup — User reads privacy explanation → selects and verifies trusted contact → reviews permissions → practices gesture.

Exception / recovery: No verified contact: cannot enable live alert; show setup action.

Normal use — Weather home and forecast work as the visible app experience. Hidden long-press is available on the agreed weather element.

Exception / recovery: Ordinary taps must not start an incident.

Activation — User performs long-press → discreet short cancellation window → incident starts unless cancelled.

Exception / recovery: Accidental trigger: cancel returns to weather; no alert should be sent.

Start / send — Request latest available location → attempt to send incident alert to approved contact → display delivery state.

Exception / recovery: Permission denied or offline: show NOT SENT / location unavailable; retry when possible.

Active — Share location only during incident; show latest timestamp, accuracy and status. Contact sees alert, map and incident timeline.

Exception / recovery: Stale location must be labelled; delivery receipt is not the same as human acknowledgement.

End — User selects End → confirmation → stop sharing → mark incident ended and update contact if possible.

Exception / recovery: If offline, queue status update and clearly show pending sync.

2. State and recovery checklist

- Idle → gesture detected → cancellation window → starting → sending → active → ending → ended.
- If the user cancels during the cancellation window: return to Idle; do not send an alert.
- If no contact is verified: block alert setup and explain how to add one.
- If location is denied: show “Location unavailable”; do not display an invented or old live position.

- If offline or sending fails: show “Alert not sent” or “Delivery unconfirmed”; retry only when safe and supported.
- If contact opens an alert: distinguish “delivered” from “contact acknowledged” (the latter needs an explicit action).
- When ending: stop incident sharing and show whether the final update was sent.

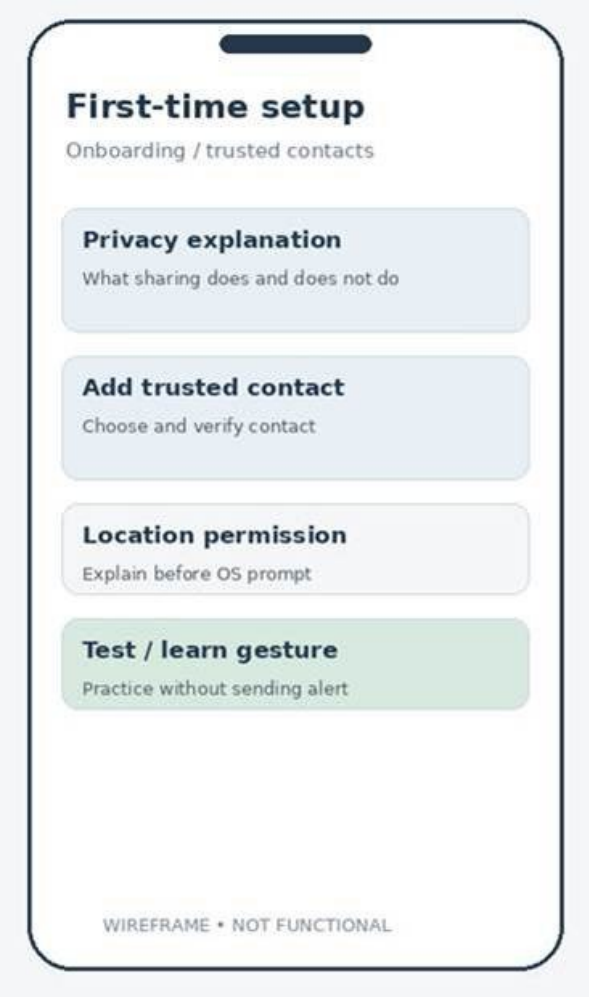
3. Low-fidelity wireframes

The screens below cover the weather landing page, onboarding, activation, incident states, trusted-contact dashboard and errors. Arrows/transitions are defined in section 1.

Weather home

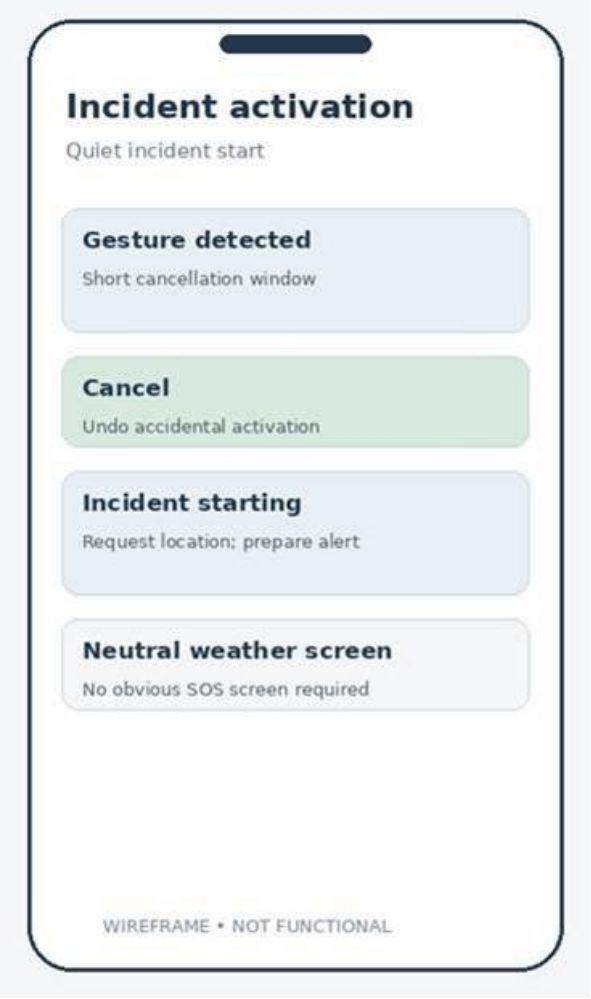


First-time setup



Incident activation

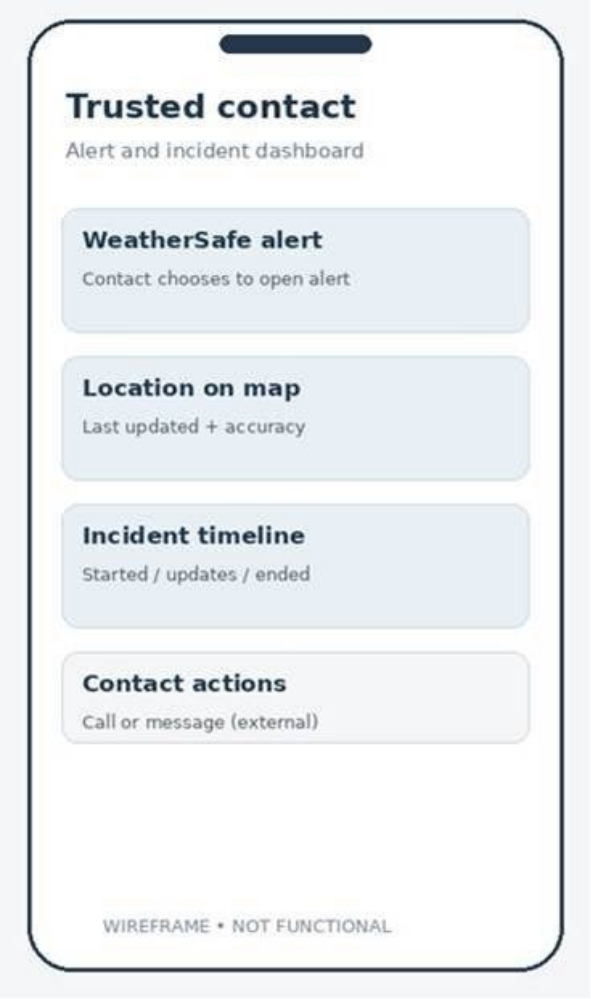
Active incident

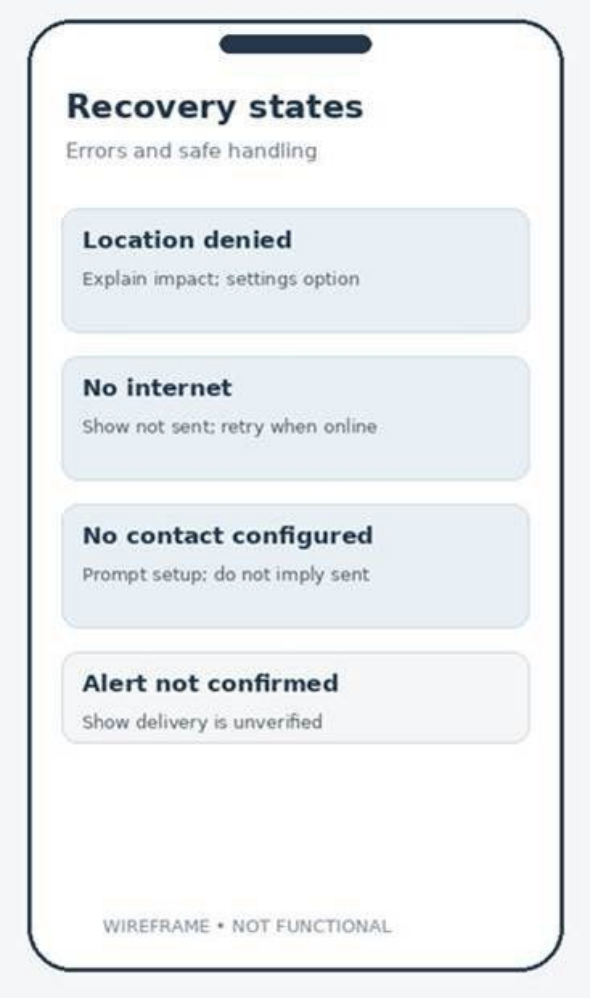


Trusted contact

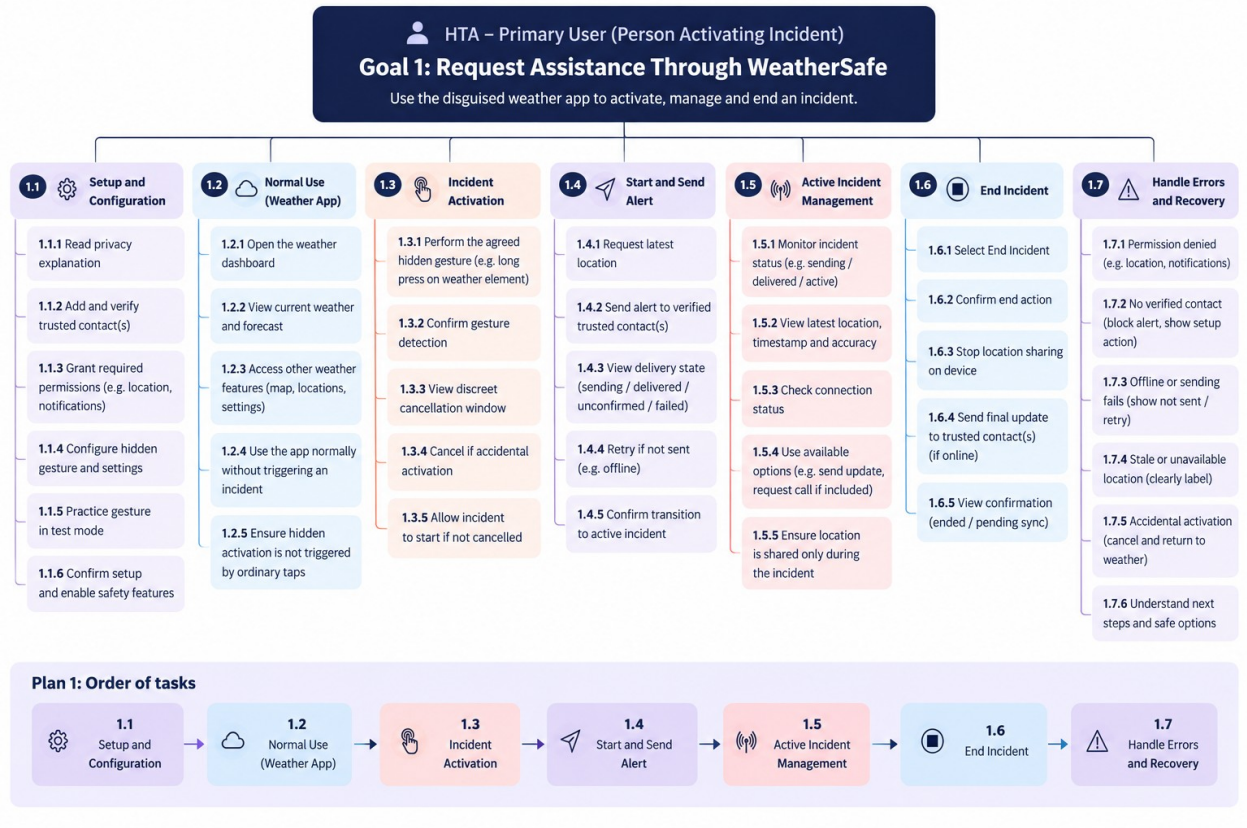


Recovery states


Trusted contact
Alert and incident dashboard
WeatherSafe alert
Contact chooses to open alert
Location on map
Last updated + accuracy
Incident timeline
Started / updates / ended
Contact actions
Call or message (external)
WIREFRAME • NOT FUNCTIONAL


Recovery states
Errors and safe handling
Location denied
Explain impact; settings option
No internet
Show not sent; retry when online
No contact configured
Prompt setup; do not imply sent
Alert not confirmed
Show delivery is unverified
WIREFRAME • NOT FUNCTIONAL

HTA



4. Security and Privacy Review

Reviewer: David Moses-Ailemen Ibukunoluwa

- **Purpose**

This review examines WeatherSafe's proposed interaction flows and wireframes to identify security and privacy risks before implementation. It covers trusted-contact pairing, access to incident information, location sharing, data retention and cancellation.

The proposed flow already includes safeguards such as contact verification, a cancellation window and location sharing during active incidents only. The requirements below clarify how these controls should behave.

- **Risk and Required Controls**

Risk	Required Control	Suggested Wireframe Update

Incorrect or unapproved trusted contact receives sensitive alerts.	Pairing must require contact verification and the user's approval	Show **Pending, Verified,** and **Removed** contact states. Include confirmation before approving or replacing a contact.
Unauthorized access exposes a user's location or incident history.	The system must check that the person requesting incident information is authorized for that specific incident.	Include a contact authentication or secure verification step and an **Access denied or expired screen.**
Accidental activation sends an unwanted alert.	Ordinary taps must not activate an incident. Cancelling within the cancellation window must prevent alert sending and discard queued work for that activation.	Show a discreet cancellation control and return to the weather screen when cancelled.
Location sharing continues after an incident ends.	Ending an incident must stop location collection and sharing on the device immediately, including when offline.	Explain that ending stops sharing. Show Sharing stopped separately from End update pending when the contact cannot yet be notified
Sensitive information is stored indefinitely	Define retention periods for location points, incident records and delivery logs. Automatically delete expired data and provide a way to request deletion.	Add a privacy explanation describing what is stored, why, for how long, and how users can request deletion.
Offline retries send an obsolete alert after cancellation on ending	Before retrying, check that the incident is still valid and the contact remains approved. Cancelled activations must never be retried, ended incidents must not restart live sharing.	Distinguish **Alert not sent, Delivery unconfirmed,** and **Incident ended.** Explain what remains queued.
Misleading status information gives users false confidence.	Distinguish sending, delivery and explicit human acknowledgement. Label location timestamps and accuracy; never present an old position as live.	Display separate delivery and acknowledgement states. Show **Last updated**, location accuracy and **Location unavailable** where appropriate.

- **Requirements for the Architecture**

The implementation must enforce permissions on the server rather than relying only on hidden buttons on screens. Sensitive data must use encrypted connections, and location details or access tokens must not appear in ordinary application logs.

Incident-handling must also prevent duplicate alerts and conflicting action. If cancellation occurs while an activation is being prepared, the cancelled activation must not subsequently send an alert. Once an alert has already been sent, ending the incident cannot retract information the contact has already received.

- **Outcome**

The proposed flow provides a useful starting point, but contact access, retention periods and cancellation behavior need clearer requirements before implementation.

The above controls should be incorporated into the relevant mock-ups and linked to development tasks. **This review identifies design requirements; completion still requires evidence that the team has applied them to the mock-ups.**

5. Trusted-contact interaction flow

This additional journey complements Section 1. The trusted contact is an independent user who must consent to their role, authenticate to access incident-specific information, acknowledge alerts deliberately, and understand when information is delayed or no longer available.

Stage	Trusted-contact actions	System response / exception
Invitation and consent	Receive invitation → review what an incident may disclose → verify identity → accept or decline.	Pending contacts have no incident access. A declined, removed or expired invitation cannot grant access.
Contact home	Open trusted-contact dashboard → review	Show an empty state when no incidents exist; prompt

	approved connections and active incidents.	reauthentication if required.
Alert received	Receive notification → open incident → complete authentication / authorisation check.	A notification is not proof of delivery or acknowledgement; denied/expired access must show a safe error.
Review incident	Read incident state and latest location → inspect timestamp, accuracy and timeline.	Show “Location unavailable” or “Last known” rather than presenting stale data as live.
Acknowledge	Select “Acknowledge alert” deliberately.	Record acknowledgement separately from notification delivery; show pending confirmation if offline.
Respond and monitor	Choose call or message action → monitor incident updates and location changes.	External call/message does not automatically mean the user is safe; updates may be delayed.
Incident ends	Receive end notification → view ended status and sharing-stopped information.	If end update has not synchronised, show last confirmed status and its timestamp, not a false assurance.
Exit / history	Return to dashboard → view permitted recent incident information.	Access remains incident-scoped and subject to retention, removal and expiry rules.

6. Connected interaction states and hand-offs

Contact verification: Primary user invites a contact; contact explicitly accepts; only then does the contact become Verified. Removing approval revokes future incident access.
Activation and cancellation: A hidden long-press starts a short cancellation window. If cancelled in time, no alert is sent and no trusted-contact incident is created.

Sending, delivery and acknowledgement: Primary user sees Sending / Not sent / Delivery unconfirmed / Delivered. Contact opening an alert is not equivalent to acknowledgement; an explicit Acknowledge action produces a separate status.
Location and timeline: Both sides see the latest available update with timestamp and accuracy. Stale or missing location is clearly marked. Sharing occurs only during an active incident.
Ending and delayed sync: Primary user ending an incident stops local sharing immediately, even offline. The trusted contact sees Ended only after an end update is received; otherwise the last confirmed state remains labelled with its timestamp.
Retries and revocation: Retries must confirm the incident is still active and the contact remains authorised. Cancelled or ended incidents must never trigger a fresh alert or resume location sharing.

7. Additional trusted-contact low-fidelity wireframes



8. Trusted-contact hierarchical task analysis (HTA)

